The Journal of Open Source Software

PyLops-MPI - MPI Powered PyLops with mpi4py

Rohan Babbar [1], **Matteo Ravasi** [2], and **Yuxi Hong** [3]

1 Computer Science and Engineering, Cluster Innovation Center, University of Delhi, Delhi, India. **2** Earth Science and Engineering, Physical Sciences and Engineering (PSE), King Abdullah University of Science and Technology (KAUST), Thuwal, Kingdom of Saudi Arabia. **3** Lawrence Berkeley National Laboratory, Berkeley, California, United States of America.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 09 October 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Linear algebra operations and inverse problems represent the cornerstone of numerous algorithms in fields such as image processing, geophysics, signal processing, and remote sensing. This paper presents PyLops-MPI, an extension of the PyLops framework, specifically designed to enable distributed computing in the solution of large-scale inverse problems in Python. PyLops-MPI provides functionalities to parallelize any combination of PyLops operators with different reduction patterns, drop-in replacements for several PyLops operator that require changes in their inner working to be amenable to distributed computing, and distributed solvers. By leveraging the Message Passing Interface (MPI) standard, the presented library can effectively unleash the computational power of multiple nodes (or ranks), enabling users to efficiently scale their inversion problems with minimal code modifications compared to their single-node equivalent PyLops codes.

Statement of need

As scientific datasets grow and the demand for higher resolution increases, the need for distributed computing alongside matrix-free linear algebra becomes more critical. Nowadays, it is in fact common for the size of models and datasets to exceed the memory capacity of a single machine—making it difficult to perform computations efficiently and accurately at the same time. However, many linear operators used in scientific inverse problems can be usually decomposed in a series of computational blocks that, although being resource-intensive, can be effectively parallelized; this further emphasizes the necessity for a distributed computing approach to inverse problems.

When addressing distributed inverse problems, we identify three distinct families of problems:

- **1. Fully distributed models and data**: both model and data are distributed across nodes, with minimal communication during the modeling process. Communication mainly occurs in the solver when dot products or regularization terms (i.e., Laplacian) are applied. In this scenario, each node can easily handle a portion of the model and data when applying the modelling operator and its adjoint.
- **2. Distributed data, model available on all nodes**: in this case, data is distributed across nodes while the model is available on all nodes. Communication is required during the adjoint pass when models produced by each node need to be summed together, and in the solver when performing dot products on the data vector.
- **3. Model and data available on all nodes**: here, communication is confined to the operator, with nodes possessing the same copies of data and model master. All nodes then perform certain computations in the forward and adjoint pass of the operator and no communication is required in the solver.

MPI for Python (also known as mpi4py (Dalcin & Fang, 2021) provides Python bindings for the MPI standard, and allows Python applications to exploit the power of multiple processors on workstations, clusters and supercomputers. Recent updates to mpi4py (version 3.0 and above) have simplified its usage, enabling more efficient data communication between nodes and processes. Some projects in the Python ecosystem, such as mpi4py-fft (Mortensen et al., 2019), mcdc (Morgan et al., 2024), and mpi4jax (Häfner & Vicentini, 2021), utilize mpi4py to extend their capabilities to distributed computing ultimately improving the efficiency and scalability of the exposed operations.

PyLops-MPI is built on top of PyLops(M. Ravasi & Vasconcelos, 2020) and utilizes mpi4py to enable the solution of large scale problems in a distributed and parallelized manner. PyLops-MPI offers an intuitive API that allows users to easily scatter and broadcast data and models across different nodes or processors, enabling to perform various mathematical operations on them (e.g., summmation, subtraction, norms) in a distributed manner. Moreover, it provides a suite of MPI-powered linear operators and linear solver and is designed in a flexible way, allowing any user to easily add custom operators and solvers tailored to their specific needs.

In summary, what sets PyLops-MPI apart from other libraries is its ease of use in creating MPI Operators, facilitating efficient integration between mpi4py and PyLops. This enables users to solve large-scale, complex inverse problems without the risk of data leaks or the need to manage MPI requirements themselves.

Software Framework

PyLops-MPI introduces MPI support to PyLops by providing an efficient API for solving linear problems through parallelization using the mpi4py library. This library is designed to tackle large-scale linear inverse problems that are difficult to solve using a single process (due to either extremely high computational cost of memory requirements).

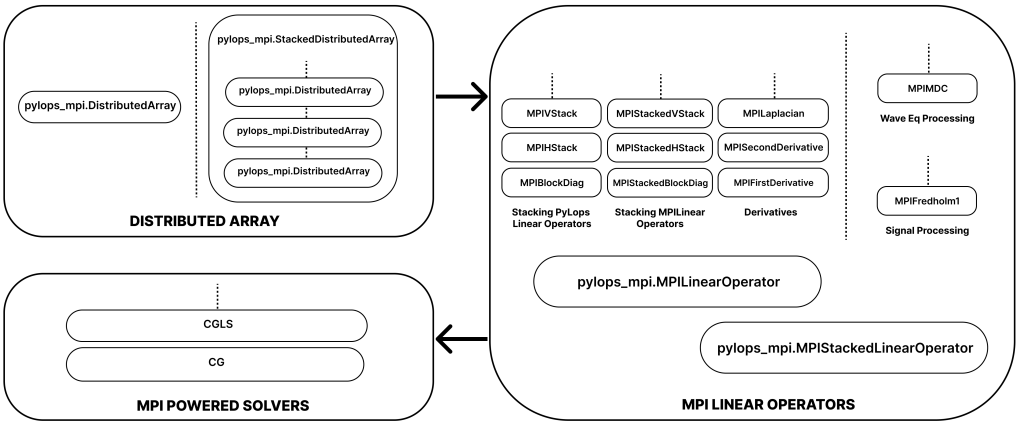

Figure 1: Software Framework representation of the `PyLops-MPI` API.

The main components of the library include:

DistributedArray

The `pylops_mpi.DistributedArray` class serves as the fundamental array class used throughout the library. It enables the partitioning of large NumPy(Harris et al., 2020) or CuPy(Okuta et al., 2017) arrays into smaller local arrays, which can be distributed across different ranks. Additionally, it allows for broadcasting the NumPy or CuPy array to multiple processes.

The DistributedArray supports two types of partitions through the **partition** attribute: `Partition.SCATTER` distributes the data across all ranks, allowing users to specify how much load each rank should handle, while `Partition.BROADCAST` creates a copy of the data and distributes it to all ranks, ensuring that the data is replicated on each rank and kept consistent across the entire duration of a code.

Furthermore, various basic mathematical functions can be applied to one or more DistributedArray objects:

- Add (+) / Subtract (-): Adds or subtracts two DistributedArrays.
- Multiply (*): Multiplies two DistributedArrays.
- Dot-product (@): Calculates the dot product by flattening the arrays, resulting in a scalar value.
- Conj: Computes the conjugate of the DistributedArray.
- Norms: Calculates the vector norm along any specified axis.
- Copy: Creates a deep copy of the DistributedArray.

PyLops-MPI also provides a way to stack a series of `pylops_mpi.DistributedArray` objects using the `pylops_mpi.StackedDistributedArray` class and perform mathematical operations on them.

MPILinearOperator

`pylops_mpi.MPILinearOperator` is the base class for all MPI linear operators, allowing users to create new operators performing matrix-vector products on `pylops_mpi.DistributedArray` objects, which can be later coupled with any PyLops-MPI linear solver. To create a new MPILinearOperator, users need to subclass the `pylops_mpi.MPILinearOperator` parent class and specify the **shape** and **dtype**. The **_matvec** and **_rmatvec** methods should also be implemented for the forward and adjoint operations.

MPIStackedLinearOperator

`pylops_mpi.MPIStackedLinearOperator` represents a second level of abstraction in the creation of MPI-powered linear operators; it allows users to create a stack of MPILinearOperator objects, where the different operators are invoked sequentially but each operator runs in a distributed fashion. The `pylops_mpi.MPIStackedLinearOperator` has the ability to perform matrix-vector products with both `pylops_mpi.DistributedArray` and `pylops_mpi.StackedDistributedArray`. Similar to `pylops_mpi.MPILinearOperator`, users need to subclass the `pylops_mpi.MPIStackedLinearOperator` parent class and specify the **shape** and **dtype**. The **_matvec** method should be implemented for the forward pass, while the **_rmatvec** method should be used for the adjoint pass.

HStack, VStack, BlockDiag Operators

One of the main features of PyLops is the ability to create any combination of linear operators in a simple and expressive manner. Three key design patterns to combine linear operators are: i) horizontal stacking, ii) vertical stacking, iii) diagonal stacking. PyLops-MPI follows the same rationale and provides distributed versions of such operators. More specifically, `pylops_mpi.MPIBlockDiag` allows one to run multiple PyLops operators in parallel over different processes, each acting on a portion of the model and data (see family 1). On the other hand, `pylops_mpi.MPIVStack` allows one to run multiple PyLops operators in parallel, each acting on the entire model in forward mode; the adjoint of such operators is instead applied to different portions of the data vector and the individual outputs are then sum-reduced (see family 2). Finally, `pylops_mpi.MPIHStack` is simply the adjoint of `pylops_mpi.MPIVStack`.

DRAFT

Halo Exchange

PyLops-MPI Linear Operators typically utilize halo exchange to facilitate interchange of portions of the model/data betweem consecutive ranks. Whilst users are encouraged toensure that the local data shapes at each rank are consistent to enable matrix-vector products without requiring external communication, in some cases this is not possible; instead, if the local shapes of the data and model at each rank do not match during these operations, the operator itself is tasked to perform a halo exchange, transferring boundary data cells (commonly referred to as "ghost cells") to/from neighboring processes. This ensures that the model vector and data vector shapes at each rank are correctly aligned for the operation. Consequently, this data transfer enables efficient local computations without the need for explicit inter-process communication, thereby avoiding heavy communication overhead.

MPI-powered Solvers

PyLops-MPI offers a small subset of PyLops linear solvers, which can deal with **DistributedArray** and **StackedDistributedArray** objects. Internally, the solvers use the different mathematical operations implemented for such classes alongside calling the forward and adjoint passes of the operator itself. The currently available solvers can be found within the submodule `pylops_mpi.optimization`.

Use Cases

In the following, we briefly discuss three different use cases in geophysical inverse problems that fall within the 3 different families of problems previously discussed. More specifically:

- *1. Seismic Post-Stack Inversion* represents an effective approach to quantitative characterization of the subsurface (Matteo Ravasi & Birnie, 2021) from seismic data. In 3D applications, both the model and data are three-dimensional (2 spatial coordinates and depth/time). PyLops-MPI solves this problem by distributing one of the spatial axes across different ranks, allowing matrix-vector products and inversions to take place at each rank, which are later gathered to obtain the inverted model. Usually communication happens because of the introducing of regularization terms that ensure the solution to be smooth or blocky.
- *2. Least-Squares Migration (LSM)* is the process of explaining seismic data via a Born modelling engine to produce an image of the subsurface reflectivity(Nemeth et al., 1999). PyLops-MPI tackles this problem by distributing all of the available sources across different MPI ranks. Each rank applies the expensive Born modeling operator for a subset of sources with the broadcasted reflectivity. The resulting data is therefore scattered across the different ranks, and inversion is again perfromed using one of MPI-Powered solvers to produce the desired subsurface reflectivity image.
- *3. Multi-Dimensional Deconvolution (MDD)* is a powerful technique used at various stages of the seismic processing value chain to create datasets deprived of overburden effects(Matteo Ravasi et al., 2022). PyLops-MPI addresses this large-scale inverse problem by splitting the kernel of the so-called Multi-Dimensional Deconvolution (MDC) operator across ranks, such that each process can perform a portion of the batched matrix-vector (or matrix-matrix) multiplication required by such an operator. Here, both the model and data are available on all ranks for the entire inverse process.

Finally, we anticipate that similar patterns can be found in many other inverse problems in different disciplines and therefore we foresee a wide adoption of the PyLops-MPI frameworks in other scientific fields.

DRAFT

References

Dalcin, L., & Fang, Y.-L. L. (2021). mpi4py: Status update after 12 years of development. *Computing in Science & Engineering*, *23*(4), 47–54. https://doi.org/10.1109/MCSE.2021.3083216

Häfner, D., & Vicentini, F. (2021). mpi4jax: Zero-copy MPI communication of JAX arrays. *Journal of Open Source Software*, *6*(65), 3419. https://doi.org/10.21105/joss.03419

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Morgan, J. P., Variansyah, I., Pasmann, S. L., Clements, K. B., Cuneo, B., Mote, A., Goodman, C., Shaw, C., Northrop, J., Pankaj, R., Lame, E., Whewell, B., McClarren, R. G., Palmer, T. S., Chen, L., Anistratov, D. Y., Kelley, C. T., Palmer, C. J., & Niemeyer, K. E. (2024). Monte carlo / dynamic code (MC/DC): An accelerated python package for fully transient neutron transport and rapid methods development. *Journal of Open Source Software*, *9*(96), 6415. https://doi.org/10.21105/joss.06415

Mortensen, M., Dalcin, L., & Keyes, D. E. (2019). mpi4py-fft: Parallel fast fourier transforms with MPI for python. *Journal of Open Source Software*, *4*(36), 1340. https://doi.org/10.21105/joss.01340

Nemeth, T., Wu, C., & Schuster, G. T. (1999). Least-squares migration incomplete reflection data. *Geophysics*, *64*(1), 208–221. https://doi.org/10.1190/1.1444517

Okuta, R., Unno, Y., Nishino, D., Hido, S., & Loomis, C. (2017). CuPy: A NumPy-compatible library for NVIDIA GPU calculations. *Proceedings of Workshop on Machine Learning Systems (LearningSys) in the Thirty-First Annual Conference on Neural Information Processing Systems (NIPS)*. http://learningsys.org/nips17/assets/papers/paper_16.pdf

Ravasi, Matteo, & Birnie, C. (2021). A joint inversion-segmentation approach to assisted seismic interpretation. *Geophysical Journal International*, *228*(2), 893–912. https://doi.org/10.1093/gji/ggab388

Ravasi, Matteo, Selvan, T., & Luiken, N. (2022). Stochastic multi-dimensional deconvolution. *IEEE Transactions on Geoscience and Remote Sensing*, *60*, 1–14. https://doi.org/10.1109/tgrs.2022.3179626

Ravasi, M., & Vasconcelos, I. (2020). PyLops - A linear-operator Python library for scalable algebra and optimization. *SoftwareX*, *11*. https://doi.org/10.1016/j.softx.2019.100361

DRAFT